

ProbOS — Month 2, Week 6

pybind11: C++ Kernel Enters the Python Package

Nisong Monyimba

July 3, 2026

Week 6 goal

Stop maintaining two parallel, disconnected implementations. Bind `BatteryCell` and `MonteCarloEngineOMP` (Week 4 C++ kernel) into Python via `pybind11`, so the 7x-faster C++ path becomes directly callable, and cross-validate it against the Python engine.

Day-by-day plan

Day 1 – `pybind11` setup

- Install `pybind11` into the project's dev environment
- Locate the exact `BatteryCell` and `MonteCarloEngineOMP` signatures from Week 4 before writing bindings

Day 2 – Bindings module

- `cpp/bindings/probos_bindings.cpp`: expose `battery_nominal_params()`, `battery_initial_state()`, `battery_forward_step()`, `MCRresult`, `MonteCarloEngineOMP` as the `probos_cpp` module
- Zero-copy NumPy array wrapping via capsule ownership

Day 3 – CMake integration + build

- `pybind11_add_module()` target in `cpp/CMakeLists.txt`, guarded by `find_package(pybind11)` so the build still succeeds without it installed
- Configure and build the extension; resolve Python interpreter/library path mismatches between the project venv and the system Python

Day 4 – Cross-validation with the Python engine

- Single-step exact agreement: `BatteryCell::forward_step()` vs `BatteryModel2Cell.forward_batch()` at `rtol=1e-10`
- 50-step accumulated-precision check

- Full-run comparison: P50 estimates agree within 10%; P05–P95 spread does NOT match, because the Week 4 C++ engine uses a simplified 5% uniform parameter perturbation, not the full `battery_priors` distributions Python draws from
- Decision: document this gap explicitly with a dedicated test that fails loudly if it silently closes or widens, rather than quietly fixing or hiding it

Day 5 – Test suite

- `python/tests/test_cpp_bindings.py`: module constants, `BatteryCell` bindings, single/multi-step cross-validation, `MonteCarloEngineOMP` bindings, cross-validation with the Python engine including the documented known-gap test
- Graceful skip via `pytest.importorskip` when `cpp/build` has not been built – verified on a fresh clone

Day 6 – CI integration

- Extend `python-tests` CI job to install `pybind11`, build the extension, and run the new binding tests
- Extend `mypy/ruff` file lists in CI to cover `particle_filter.py` and `python/pdsl/` (previously missing)

Day 7 – Study guide + commit

- `docs/study/study_guide.md` entries: `pybind11` documentation, CMake integration guide
- Full verification and push

What was actually accomplished (retrospective)

- `probos.cpp` `pybind11` extension built and importable
- Single-step numerical correctness proven to `rtol=1e-8` to `1e-10` against the Python reference – not just “runs”, genuinely verified
- Known P05–P95 spread discrepancy between C++ and Python engines documented AND tested-for explicitly, rather than silently accepted or silently fixed
- 24 new tests, all passing
- `pyproject.toml`: `pybind11` added to dev dependencies in the same commit that introduced the dependency (learned directly from the Week 4 `lark` CI-break incident)

Numbers at Week 6 close

- Python tests: 308/308 passing (284 + 24 new)
- C++ tests: 13/13 passing
- `mypy` strict: 0 errors
- `ruff`: 0 warnings

Carried into later weeks

Before continuing to Week 7 (FastAPI service layer), a full systematic audit was performed to find and fix latent problems while the surface area was still small – see the pre-Week-7 hardening pass documented in `docs/monthly_plans/month1/week4/main.tex`'s retrospective and `docs/standards/quality_standards`. This audit found several real, fixable issues (missing `--init--.py`, uncertain mypy coverage, a stale coverage floor, a bandit finding on the PDSL compiler) before they could compound under a new service layer.